DriveShare Project Report (Design Patterns Implementation)

1. Introduction

DriveShare is a peer-to-peer car rental platform inspired by services such as Turo. The system allows users to register, list vehicles, search and book cars, communicate with owners, and receive notifications regarding bookings, payments, and car updates.

The system is implemented using Flask and SQLite and follows a layered architecture with a strong focus on object-oriented design principles and design patterns to improve scalability, maintainability, and modularity.

2. Architecture Overview

The system is structured into:

- **Presentation Layer** (app.py, HTML templates)
- **Service Layer** (business logic: car_service.py, booking_service.py, etc.)
- **Pattern Layer** (patterns/ folder)
- **Data Layer** (database.py, SQLite DB)

Design patterns are used to solve specific architectural problems such as object creation, communication, state management, and secure processing.

3. Design Pattern Implementations

3.1 Builder Pattern – Car Creation

Files:

- patterns/builder.py
- car_service.py

Implementation

The Builder pattern is used to construct complex Car objects step-by-step, allowing optional attributes to be set in a readable and flexible way.

Code snippet (builder.py):

```
class Car:
  def __init__(self):
    self.owner_id = None
    self.model = None
    self.year = None
    self.mileage = None
    self.location = None
    self.price = None
    self.available = None

class CarBuilder:
  def __init__(self):
    self.car = Car()

  def set_owner(self, owner_id):
    self.car.owner_id = owner_id
    return self

  def set_model(self, model):
    self.car.model = model
    return self

  def set_year(self, year):
    self.car.year = year
```

```
        return self

    def set_mileage(self, mileage):
        self.car.mileage = mileage
        return self

    def set_location(self, location):
        self.car.location = location
        return self

    def set_price(self, price):
        self.car.price = price
        return self

    def set_available(self, available):
        self.car.available = available
        return self

    def build(self):
        return self.car
```

Usage in CarService

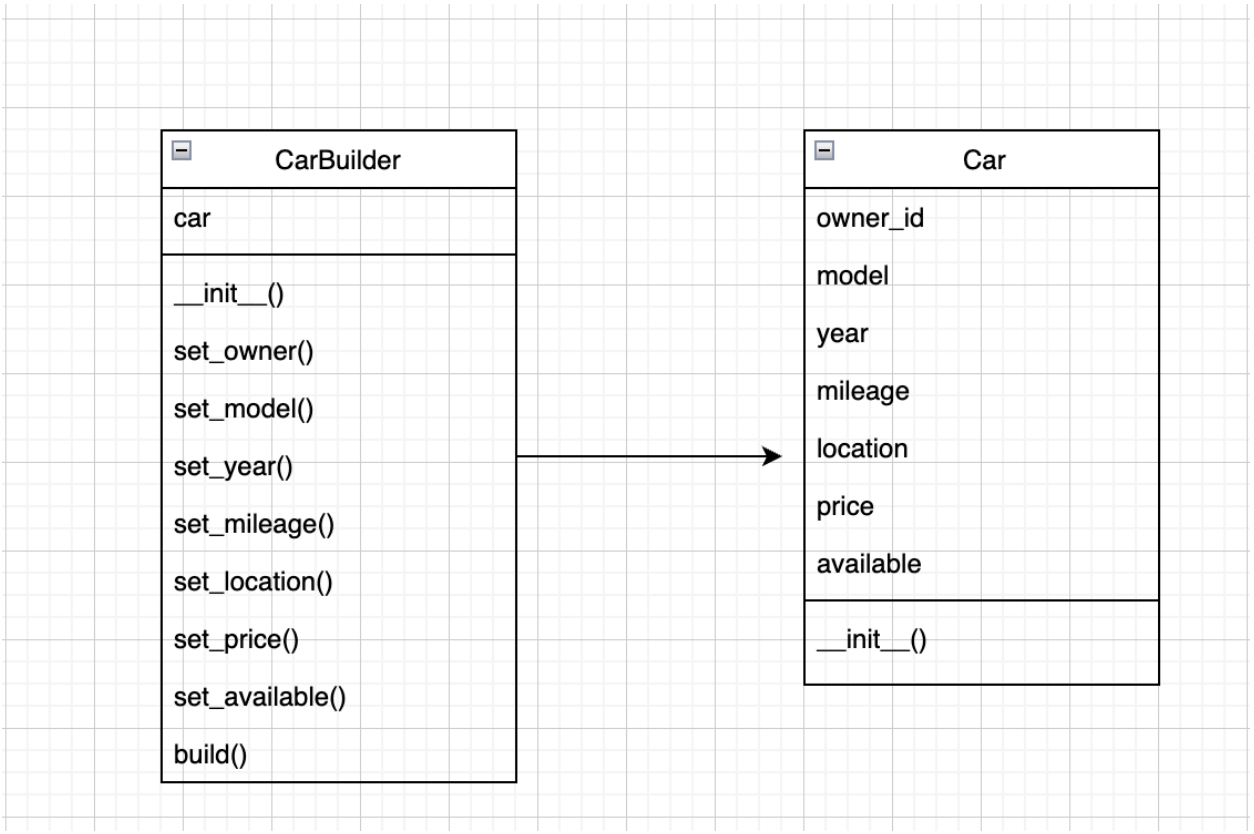
```
car = (CarBuilder()
    .set_owner(owner_id)
    .set_model(model)
    .set_year(year)
    .set_mileage(mileage)
    .set_location(location)
    .set_price(price)
    .set_available(available)
    .build())
```

Then inserted into the database.

Purpose

- Simplifies car object creation
- Supports optional attributes
- Improves readability and maintainability

Class Diagram



The class diagram illustrates how the Builder pattern separates object construction from representation. The Car class represents the final product, while CarBuilder encapsulates the construction process through a series of setter methods.

CarBuilder maintains a reference to a Car object and progressively configures it before returning the final instance via the build() method. The relationship between CarBuilder and Car is a composition relationship, as the builder internally creates and manages the Car object.

In the standard UML structure, a Director class is often included to control the building process. In this implementation, that role is handled implicitly by the CarService, which orchestrates the sequence of builder method calls.

This diagram reflects a simplified but valid Builder structure where object construction logic is fully encapsulated within the builder and invoked externally.

3.2 Singleton Pattern – Session Management

Files:

- patterns/singleton.py
- app.py

Implementation

The Singleton pattern ensures only one session manager exists across the application.

Code:

```
class SessionSingleton:
  _instance = None

  def __new__(cls):
    if cls._instance is None:
      cls._instance = super().__new__(cls)
      cls._instance.user_id = None
    return cls._instance
```

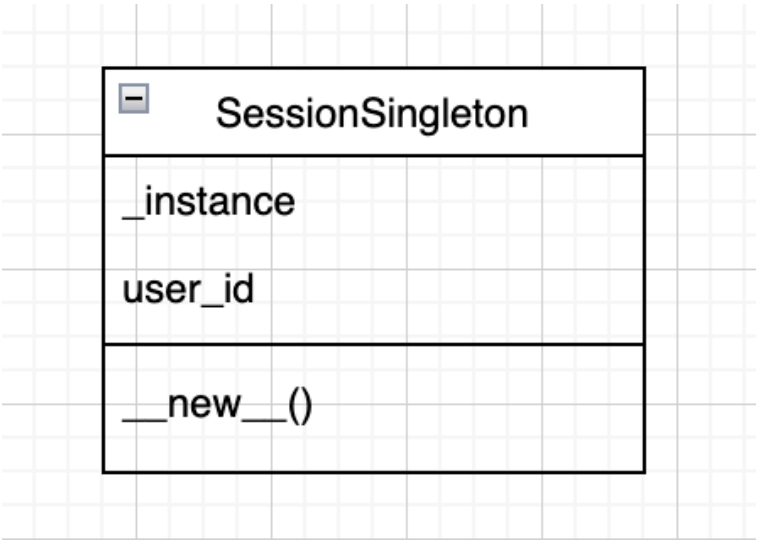
Usage in app.py

```
session_manager = SessionSingleton()

if ok:
  session["user_id"] = user["id"]
  session_manager.user_id = user["id"]
```

Purpose

- Maintains global session state
- Ensures consistent user identity across system
- Prevents multiple session instances

Class Diagram



The class diagram shows a single class, SessionSingleton, with a static instance variable (_instance) that ensures only one object is created. The key feature is the control over instantiation through the overridden __new__() method.

Unlike traditional UML diagrams that may show a private constructor, this implementation uses Python's object model to enforce the Singleton constraint. The diagram represents a self-managed lifecycle, where the class is responsible for both creation and access to its instance.

There are no external dependencies or inheritance relationships, emphasizing that the Singleton is globally accessible and independent. The user_id attribute represents shared state maintained across the system.

This structure aligns with the standard Singleton pattern where a single instance is reused throughout the application.

3.3 Observer Pattern – Watch System (Price & Availability Alerts)

Files:

- patterns/observer.py
- watch_service.py

Implementation

The Observer pattern is used to notify users when:

- car price drops
- car becomes available

Subject

```
class WatchSubject:
  def __init__(self):
    self.observers = []

  def attach(self, observer):
    self.observers.append(observer)

  def notify(self, car):
    for obs in self.observers:
      obs.update(car)
```

Observer

```
class WatchObserver:
  def __init__(self, user_id, max_price, notification_service):
    self.user_id = user_id
    self.max_price = float(max_price)
    self.notifications = notification_service

  def update(self, car):
    price = float(car["price"])
    available = int(car["available"])

    if price <= self.max_price:
      self.notifications.send(
```

```
            self.user_id,
            f"{car['model']} dropped to ${price}"
        )

    if available == 1:
        self.notifications.send(
            self.user_id,
            f"{car['model']} is now available!"
        )
```

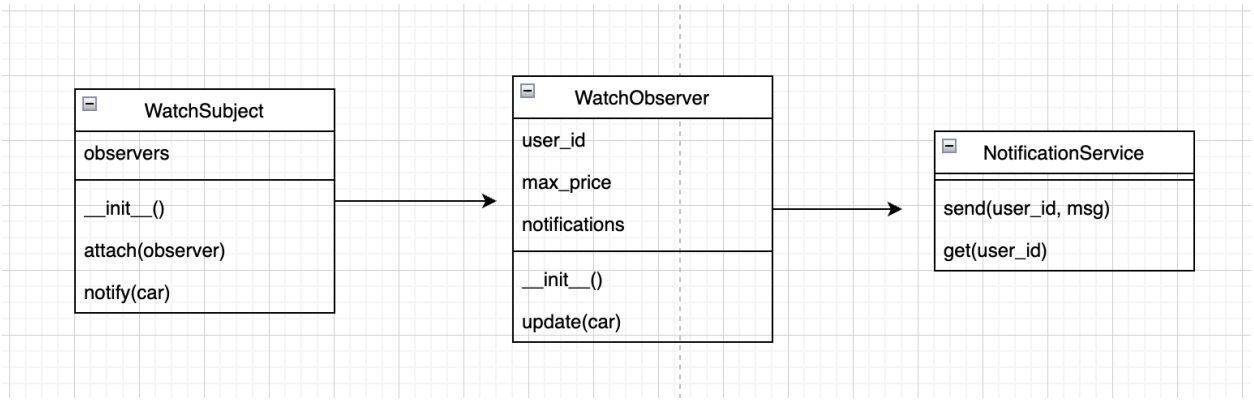
Trigger in WatchService

```
subject = WatchSubject()

for w in watches:
    observer = WatchObserver(
        w["user_id"],
        w["max_price"],
        NotificationService()
    )
    subject.attach(observer)

subject.notify(car)
```

Purpose

- Implements real-time notifications
- Allows users to “watch” cars
- Automatically triggers alerts on updates

Class Diagram



The diagram represents a one-to-many relationship between the WatchSubject and multiple WatchObserver instances. The subject maintains a collection of observers and provides methods to attach observers and notify them of changes.

Each WatchObserver contains its own state (user_id and max_price) and defines an update() method, which is invoked when the subject broadcasts a change. This demonstrates loose coupling, as the subject does not need to know the internal logic of each observer.

The NotificationService appears as a dependency used by observers to perform actions when updates occur. This highlights that observers are responsible for handling reactions, not the subject.

This structure closely follows the standard Observer UML model:

- Subject to maintain observer list
- Observer to define update interface
- Concrete Observers to implement update behavior

3.4 Mediator Pattern – System Communication Hub

Files:

- patterns/mediator.py
- app.py

Implementation

The Mediator centralizes communication between services such as messaging, payments, and system events.

Code:

```
class Mediator:
  def __init__(self, notification_service):
    self.notification_service = notification_service

  def notify(self, sender, event, user_id=None):
    message = f"[{sender}] {event}"

    if user_id:
      self.notification_service.send(user_id, message)

  def send_message(self, from_id, to_id, text):
    msg = f"FROM:{from_id}|{text}"
    self.notification_service.send(to_id, msg)

  def system_event(self, event, user_ids):
    for uid in user_ids:
      self.notification_service.send(uid, f"[SYSTEM] {event}")
```

Usage in app.py

Messaging:

```
mediator.send_message(
  session["user_id"],
  car["owner_id"],
  text
)
```

System event:

```
mediator.system_event(
  f"{car['model']} updated",
  [session["user_id"]]
)
```

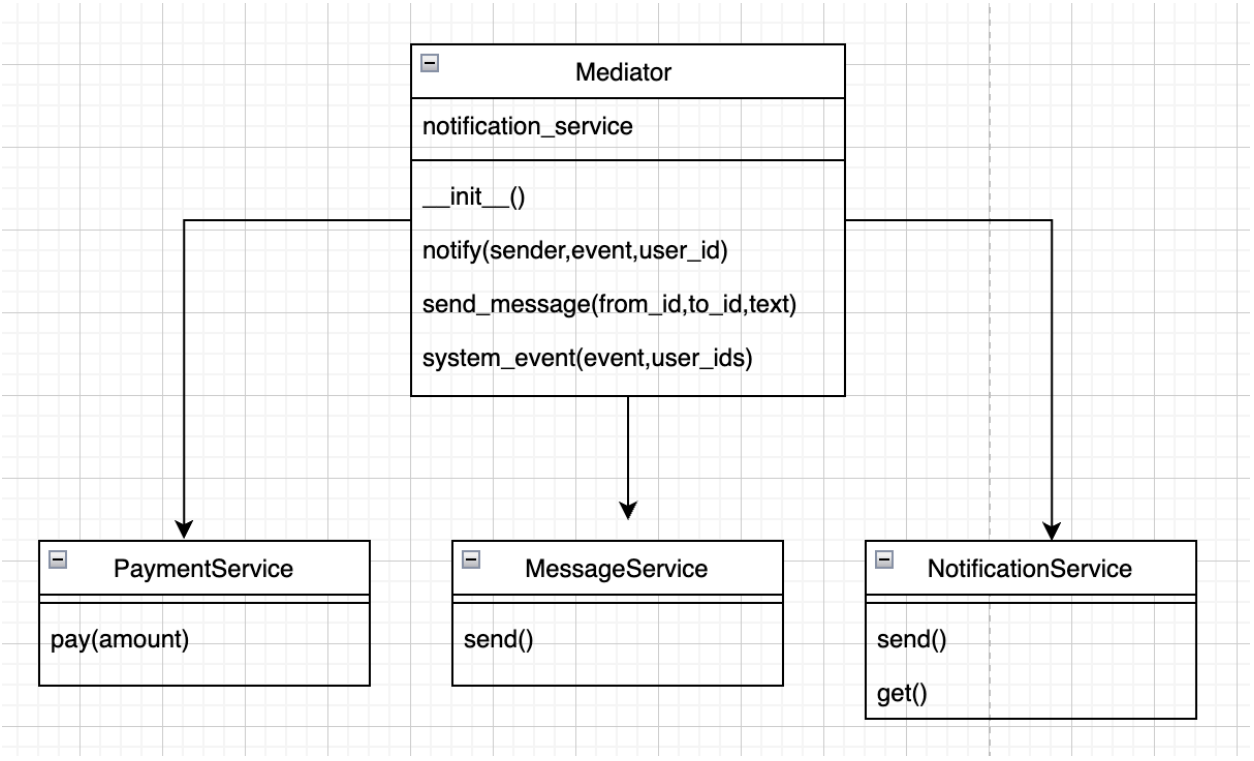
Payment notifications:

```
mediator.notify(
  "PAYMENT",
  f"You paid ${car['price']} for {car['model']}",
  session["user_id"]
)
```

Purpose

- Centralizes communication logic
- Reduces coupling between services
- Handles messaging + system notifications

Class Diagram



The class diagram centers around the Mediator class, which acts as an intermediary between different components in the system. Instead of components communicating directly, they rely on the mediator to coordinate interactions.

Unlike the classic UML diagram that explicitly shows multiple colleague classes, this implementation represents colleagues more implicitly. Components such as messaging, payment handling, and system events interact with the mediator through its methods (notify, send_message, system_event).

The mediator depends on the NotificationService, which serves as the underlying mechanism for delivering messages. This creates a structure where communication flows: Component to Mediator to NotificationService

This design simplifies relationships by reducing direct dependencies between components. While it omits explicit colleague subclasses, it still satisfies the core Mediator principle of centralized communication control.

3.5 Proxy Pattern – Payment Security Layer

Files:

- patterns/proxy.py
- payment_service.py

Implementation

The Proxy pattern controls access to payment processing.

Real Payment:

```
class RealPayment:
  def process(self, amount):
    return f"Payment of ${amount} completed"
```

Proxy:

```
class PaymentProxy:
  def __init__(self):
    self.real = RealPayment()

  def process(self, amount):
    if amount <= 0:
      return "Invalid payment amount"

    print("Proxy: validating payment...")

    return self.real.process(amount)
```

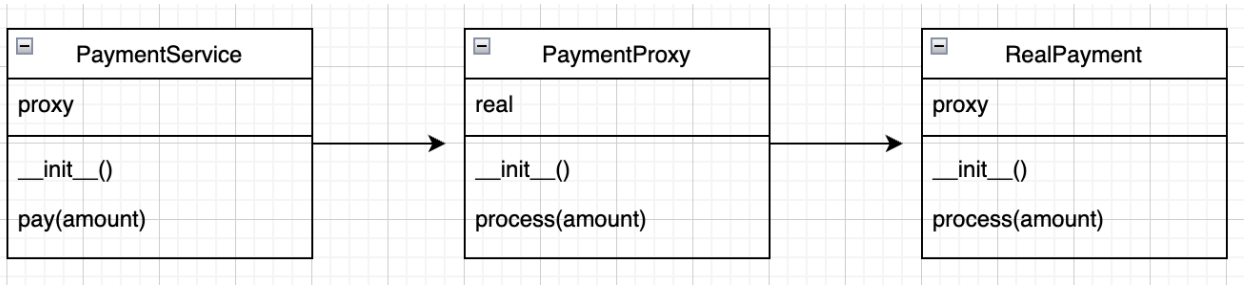
Service usage:

```
class PaymentService:
  def __init__(self):
    self.proxy = PaymentProxy()

  def pay(self, amount):
    return self.proxy.process(amount)
```

Purpose

- Adds validation layer before payment
- Simulates secure payment gateway
- Controls access to real payment logic

Class Diagram



The diagram shows two main classes: PaymentProxy and RealPayment, both exposing the same method (process). This reflects the key Proxy pattern requirement that the proxy and real subject share a common interface.

PaymentProxy holds a reference to RealPayment, forming an association relationship, and controls access by performing validation before delegating the request. The client (PaymentService) interacts only with the proxy, not the real object.

Although no formal interface is defined (which is typical in Python), the shared method signature acts as an implicit contract between the proxy and the real subject.

This structure aligns with the standard Proxy UML:

- Proxy to control access
- Real Subject to perform actual work
- Client to interact with proxy only

3.6 Chain of Responsibility – Password Recovery

Files:

- patterns/chain.py
- app.py (/recover route)

Implementation

Security questions are processed as a chain.

Base Handler:

```
class Handler:
  def __init__(self, next_handler=None):
    self.next = next_handler

  def handle(self, data):
    if self.next:
      return self.next.handle(data)
```

Chain Steps:

```
class Q1Handler(Handler):
  def handle(self, data):
    if data["input_q1"].strip().lower() == data["db_q1"].strip().lower():
      return super().handle(data)
    return False

class Q2Handler(Handler):
  def handle(self, data):
    if data["input_q2"].strip().lower() == data["db_q2"].strip().lower():
      return super().handle(data)
    return False

class Q3Handler(Handler):
  def handle(self, data):
    if data["input_q3"].strip().lower() == data["db_q3"].strip().lower():
      return True
    return False
```

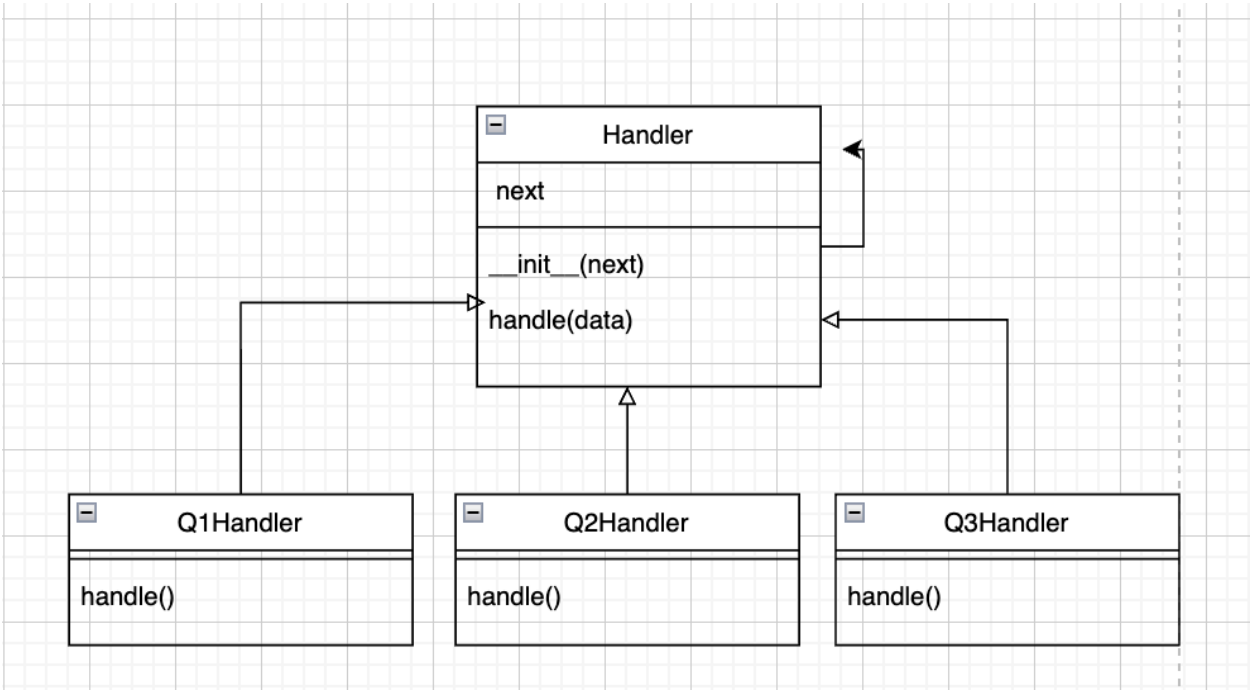
Usage in app.py

```
chain = Q1Handler(Q2Handler(Q3Handler()))

ok = chain.handle(data)
```

Purpose

- Secure multi-step authentication
- Modular verification process
- Easy to extend with more questions

Class Diagram



The class diagram represents a linear chain of handler objects connected through a next reference. The base Handler class defines the structure of the chain, including the reference to the next handler and the handle() method.

The concrete handlers (Q1Handler, Q2Handler, Q3Handler) extend the base class and override the handle() method to perform specific validation logic. Each handler either processes the request or passes it along the chain.

The relationship between handlers forms a linked structure, where each object knows only about the next one in the sequence. This creates a flexible processing pipeline.

This directly corresponds to the standard Chain of Responsibility UML:

- Handler to define interface and next reference
- Concrete Handlers to implement processing logic
- Chain to sequential link handlers

The diagram highlights how responsibility is distributed across multiple objects rather than handled by a single class.

4. Database Schema and Description

Database Overview

The DriveShare system uses an SQLite database to store users, cars, bookings, notifications, and watch preferences. The database is initialized in database.py.

Users Table

Purpose: Stores user account information and security questions.

Schema:

- id (INTEGER, Primary Key)
- email (TEXT, UNIQUE)
- password (TEXT)
- q1 (TEXT)
- q2 (TEXT)
- q3 (TEXT)

Description:
Each user registers with an email and password and must provide three security questions used for password recovery.

Cars Table

Purpose: Stores car listings created by owners.

Schema:

- id (INTEGER, Primary Key)
- owner_id (INTEGER)
- model (TEXT)
- year (INTEGER)
- mileage (INTEGER)
- location (TEXT)
- price (REAL)
- available (TEXT)

Description:
Each car is associated with an owner and includes details such as model, price, and availability. This table is used for searching and booking cars.

Bookings Table

Purpose: Stores car reservations.

Schema:

- id (INTEGER, Primary Key)
- car_id (INTEGER)
- user_id (INTEGER)
- start_date (TEXT)
- end_date (TEXT)
- price (REAL)

Description:
This table tracks all bookings made by users. The system prevents overlapping bookings for the same car.

Notifications Table

Purpose: Stores messages and system notifications.

Schema:

- id (INTEGER, Primary Key)
- user_id (INTEGER)
- message (TEXT)
- type (TEXT)

Description:
This table stores both user messages and system-generated notifications such as booking updates and price alerts.

Watches Table

Purpose: Stores user watch preferences for cars.

Schema:

- id (INTEGER, Primary Key)
- user_id (INTEGER)
- car_id (INTEGER)
- max_price (REAL)

Description:
Users can “watch” cars and specify a maximum price. This table is used by the Observer pattern to trigger notifications.

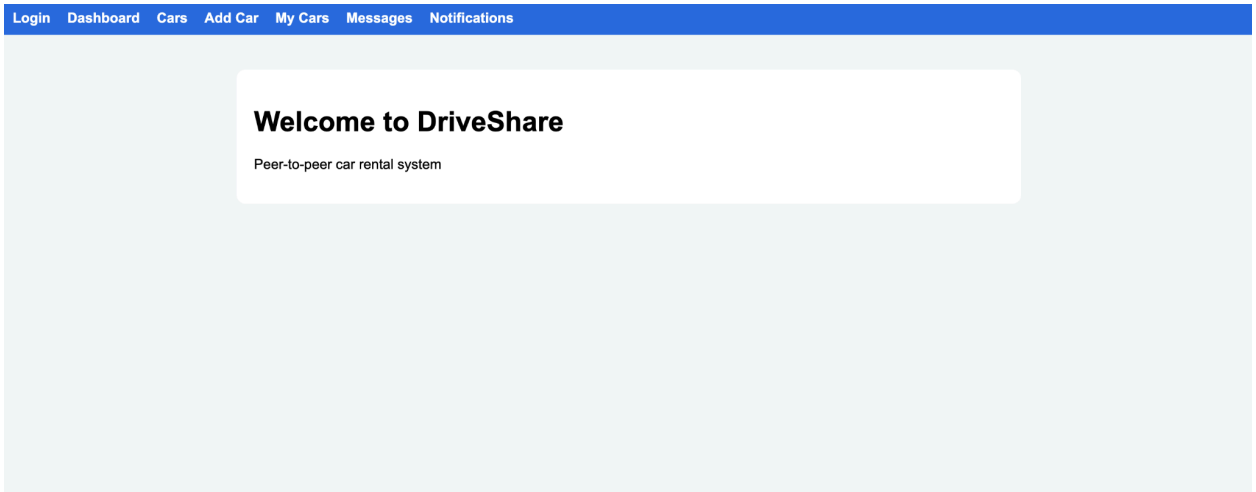
The database schema supports all core system features and integrates with the implemented design patterns, particularly the Observer pattern (watches table) and Chain of Responsibility (user security questions).

Screenshots:

Home Page

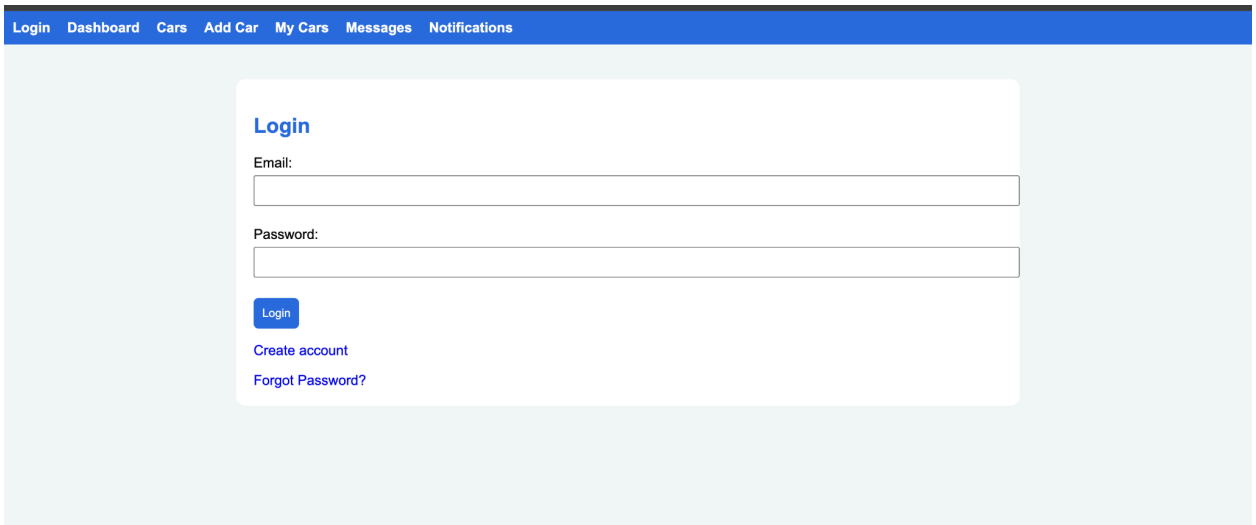
Description:

The home page serves as the entry point of the DriveShare platform. It provides navigation options for users to register or log in to the system. This page is designed to introduce the application and guide users toward authentication before accessing core features.



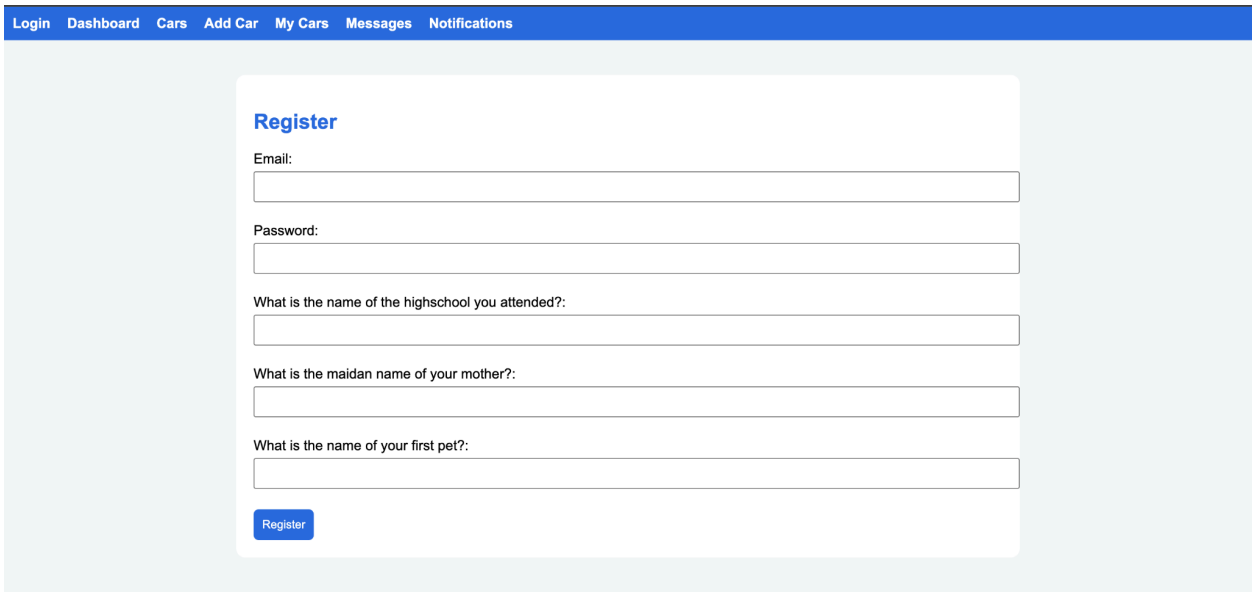
Login Page

Description:
The login page allows registered users to authenticate using their email and password. Upon successful login, the system stores the user session and redirects them to the dashboard. This page is connected to the authentication service and session management system.



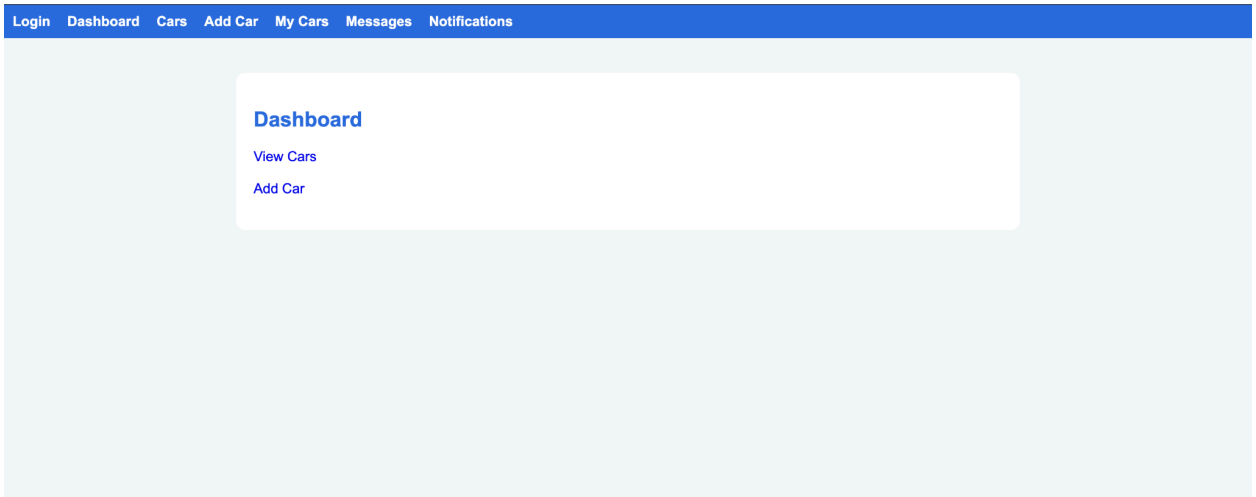
Register Page

Description:
The registration page enables new users to create an account by providing an email, password, and three security question answers. These security questions are later used in the password recovery process. The data is stored in the database upon successful registration.



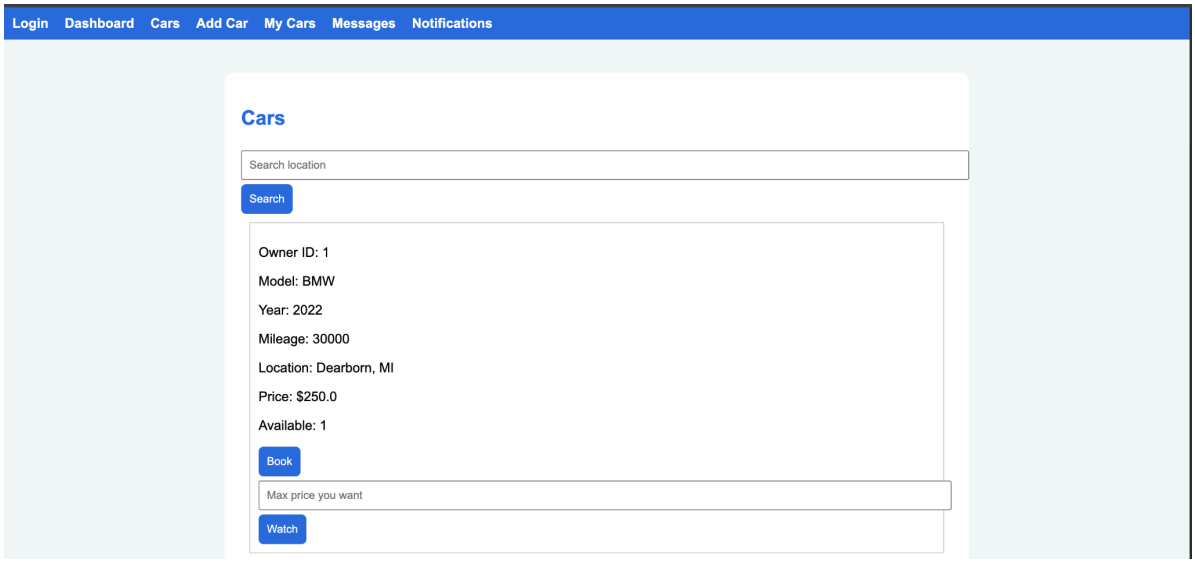
Dashboard

Description:
The dashboard acts as the main navigation hub after login. From this page, users can access features such as viewing cars, managing their listings, checking messages, and interacting with other parts of the system.



Cars List

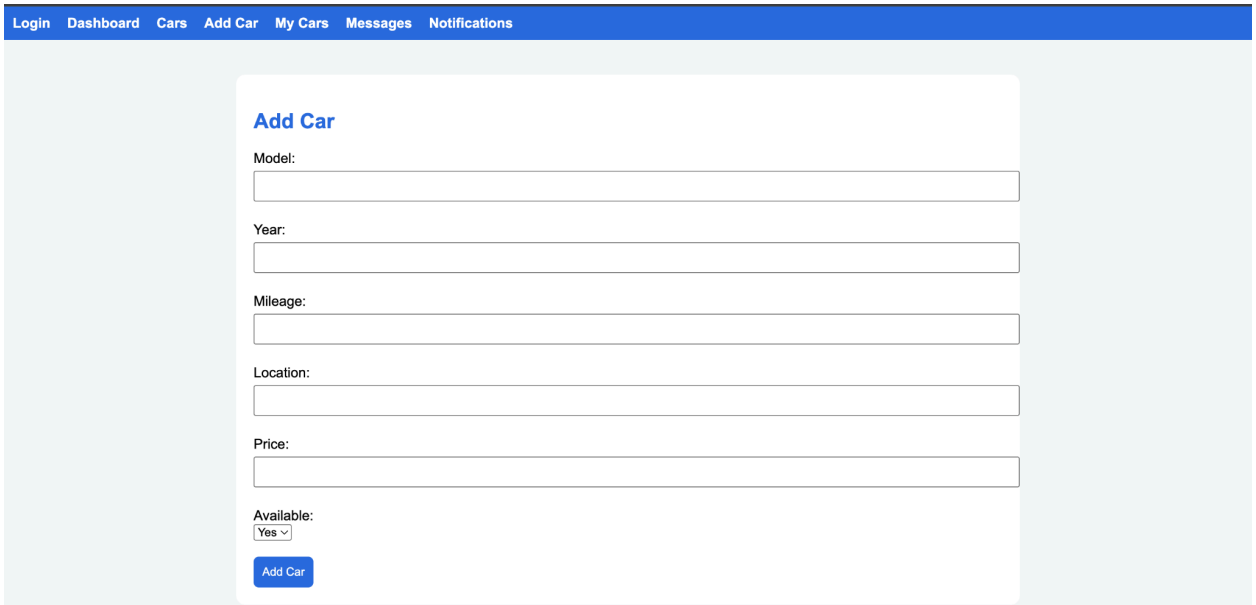
Description:
This page displays all available cars in the system. Users can search for cars based on location and view details such as price and availability. It supports the renter functionality of browsing and selecting cars for booking.



Add Page

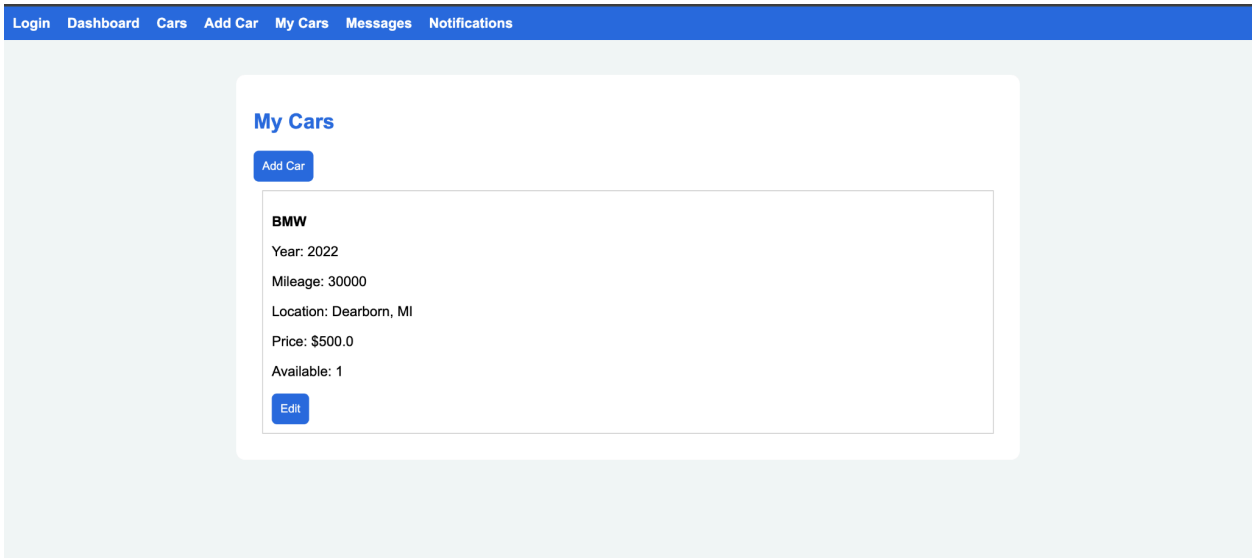
Description:
The add car page allows car owners to create a new car listing by entering details such as

model, year, mileage, location, price, and availability. The Builder pattern is used internally to construct the car object before storing it in the database.



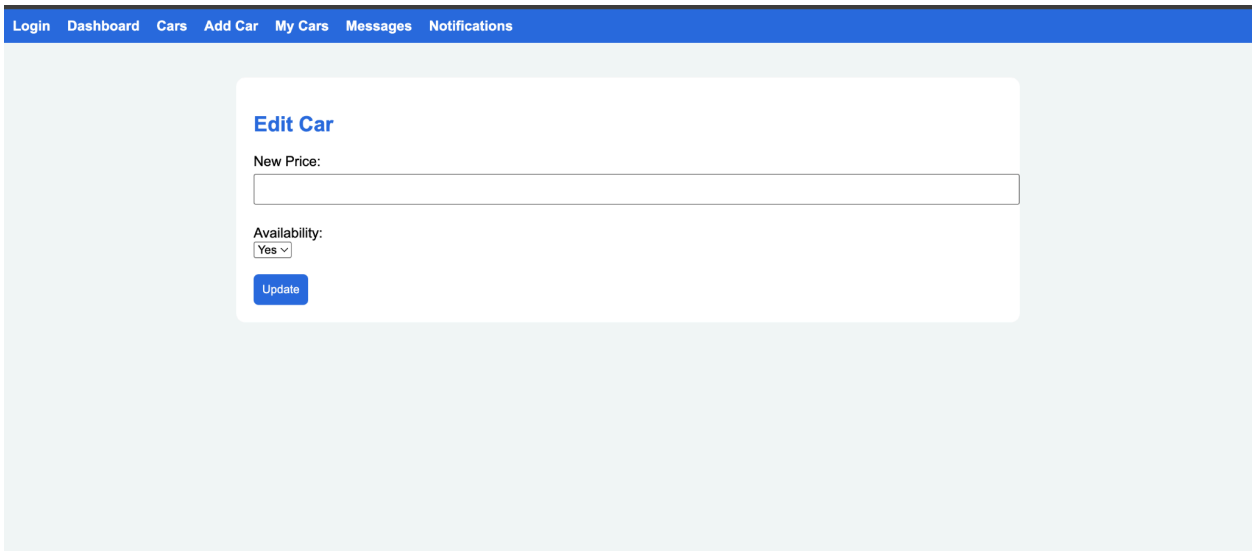
My Cars

Description:
This page displays all cars listed by the currently logged-in user. It allows owners to manage their listings, including editing car details such as price and availability.



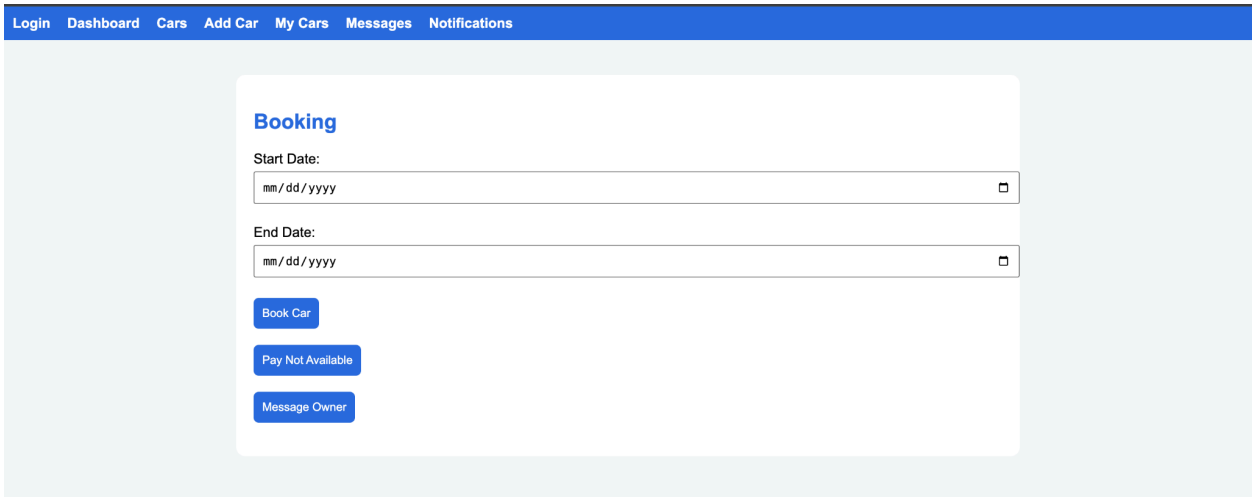
Edit Car Page

Description:
The edit car page allows owners to update the price and availability of a selected car. When changes are made, the system triggers notifications to users watching the car using the Observer pattern.



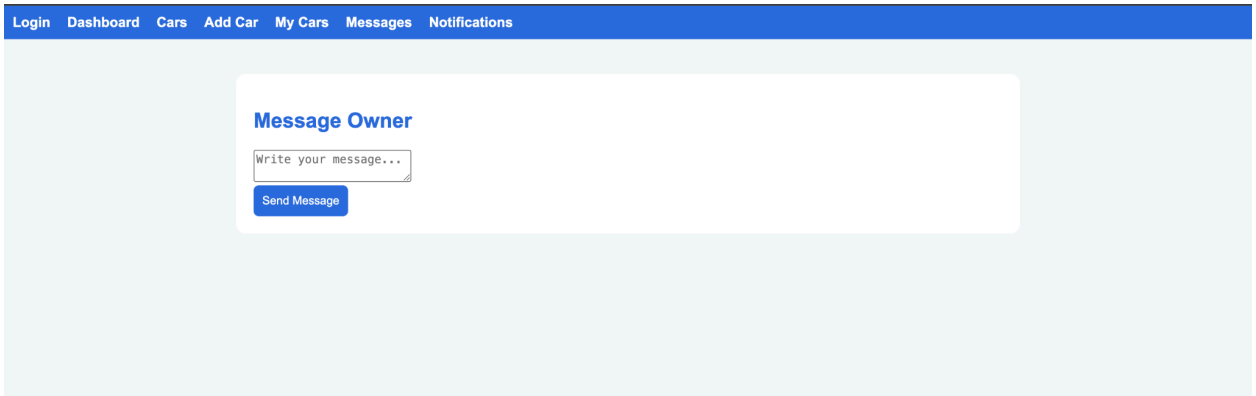
Book Car Page

Description:
This page allows renters to book a car for a selected date range. The system checks for overlapping bookings to prevent conflicts. After booking, users can proceed to payment.



Message Page

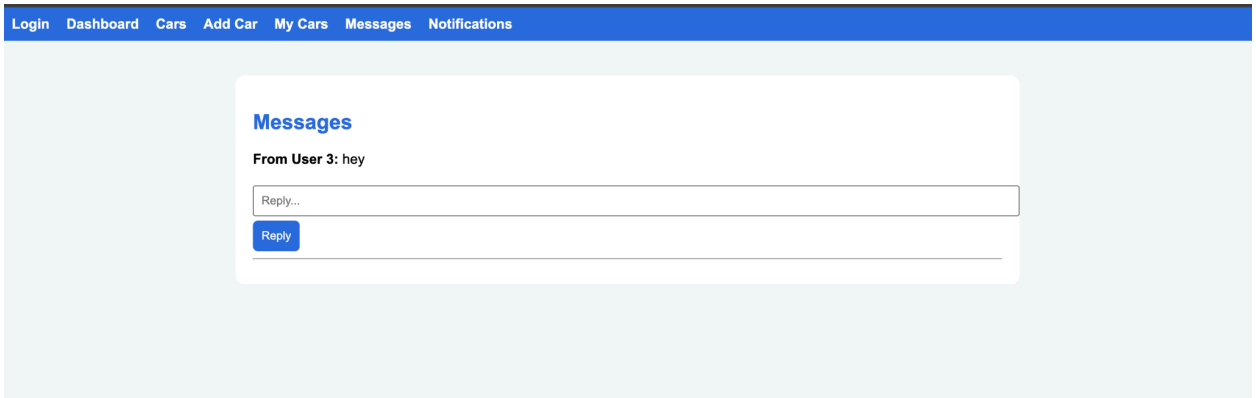
Description:
The message page allows users to send messages to car owners. Communication is handled through the Mediator pattern, which routes messages through a centralized system instead of direct interaction between users.



Messages Page

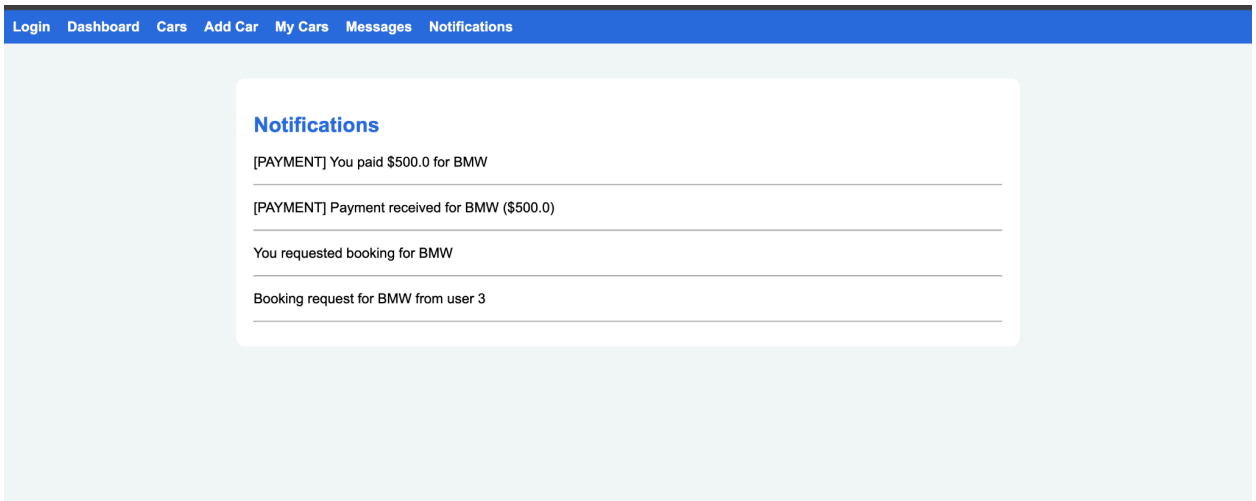
Description:
This page displays all messages and notifications received by the user. Messages include

communication from other users as well as system-generated updates such as bookings and payments.



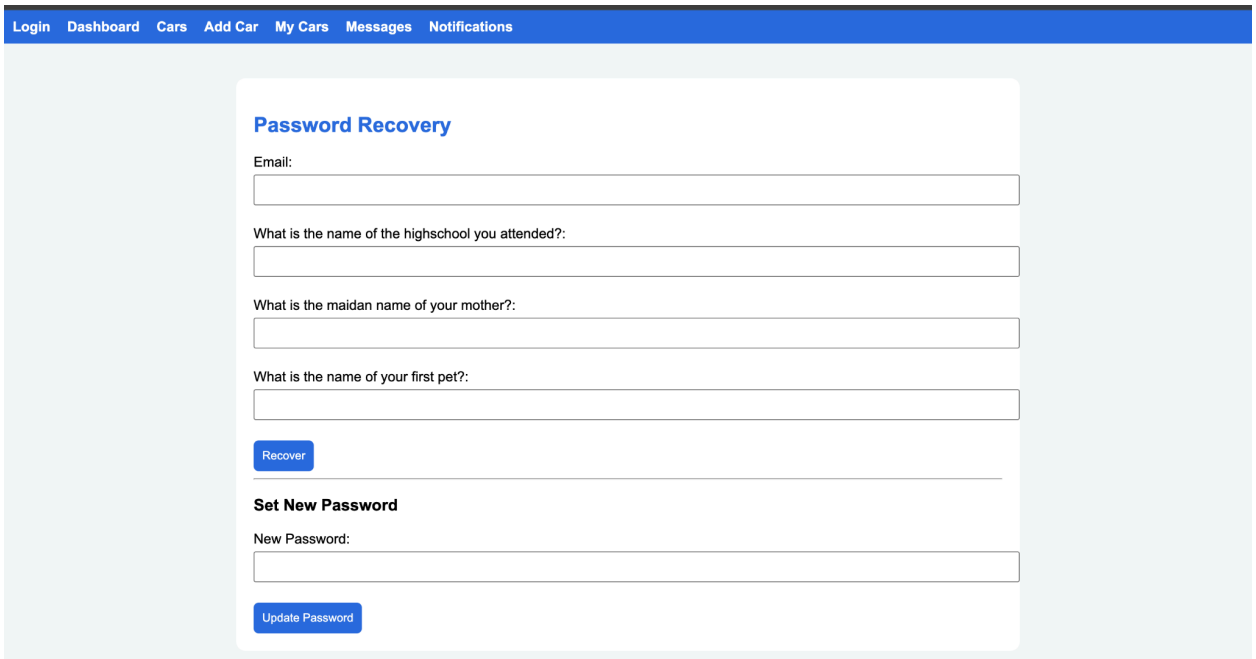
Notifications Page

Description:
The notifications page shows system-generated alerts such as booking confirmations, payment updates, and car availability changes. These notifications are triggered using the Observer and Mediator patterns.



Password Recovery Page

Description:
The password recovery page allows users to reset their password by answering three security questions. The Chain of Responsibility pattern is used to validate each answer step-by-step before allowing a password reset.



Conclusion

The DriveShare system successfully integrates six design patterns to solve real-world problems in a peer-to-peer car rental platform. Each pattern is applied to a specific subsystem, improving modularity, scalability, and maintainability.

- Builder to car creation
- Singleton to session management
- Observer to watch notifications
- Mediator to communication hub
- Proxy to payment security
- Chain to password recovery